

CS & IT ENGINEERING



Operating System

Process Synchronization

Lecture · 02

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Synchronization

Topic

Race Condition

Topic

Critical Section

Topics to be Covered



Topic

Mutual Exclusion

Topic

Progress

Topic

Bounded Waiting

Topic

Two-Process Solution for Critical Section

[NAT]

The following two functions $P1$ and $P2$ that share a variable B with an initial value of 2 execute concurrently.

$$B = 3, 4, 2$$

$P1() \{$ $\quad C = B - 1;$ $\quad B = 2 * C;$ $\}$	$P2() \{$ $\quad D = 2 * B;$ $\quad B = D - 1;$ $\}$
---	---

The number of distinct values that B can possibly take after the execution is 3.

[NAT]



Consider three concurrent processes P1, P2 and P3 as shown below, which access a shared variable D that has been initialized to 100.

P1	P2	P3
$\vdots$	$\vdots$	$\vdots$
$D = D + 20$	$D = D - 50$	$D = D + 10$
$\vdots$	$\vdots$	$\vdots$

$$D = 100$$

$$D_{\min} = X = 50$$

$$D_{\max} = Y = 130$$

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed execution are X and Y respectively, then the value of $Y - X$ is 80.

$$Y - X = 130 - 50 = 80$$

In prev. questⁿ, no. of diff. possible values of $D = \underline{7}$?

$$D + 20 - 50 + 10$$

$$D = 120$$

$$= 50$$

$$= 110$$

$$= 70$$

$$= 130$$

$$= 60$$

$$= 80$$

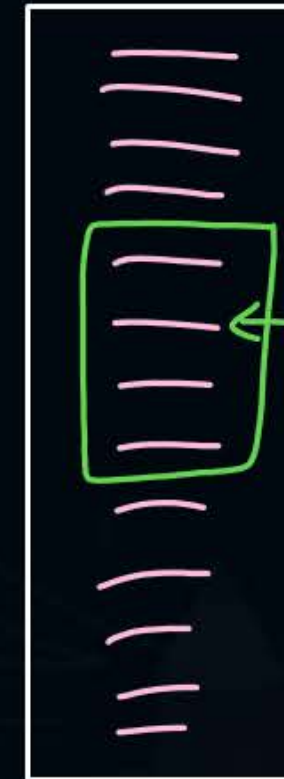


Topic : Solution of Critical Section Problem

Requirements of Critical Section problem solution:

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Process



critical
section



Topic : Solution of Critical Section Problem

Mutual Exclusion:

If one process is executing the critical section, then other process is not allowed to enter into critical section.

2 processes can not be in critical section together.



Topic : Solution of Critical Section Problem

Progress:

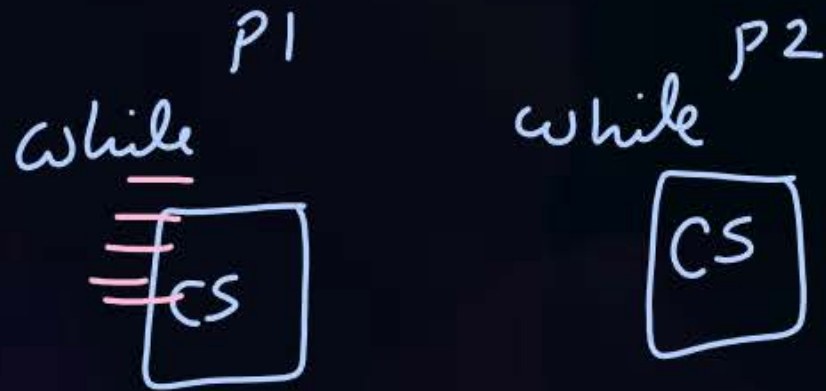
If no any process is in critical section and any process wants to enter into critical section, then the process must be allowed.



Topic : Solution of Critical Section Problem

Bounded Waiting:

If a process p_1 is executing in critical section and other process p_2 is waiting for critical section, then the waiting time of p_2 must be bounded. Which means p_1 must not enter in to critical section again and again by keeping p_2 in waiting for long.





Topic : 2-Process Solution



Entry section

→ ensure all 3 requirements

C.S.

Exit section

→ for announcement that process is out of C.S.

Solution 1

shared b/w P0 & P1

if c.s. is free
or locked

Boolean lock=false;

P0

int lock = 0

while(true)

{

while(lock);

lock=true;

//CS

lock=false;

RS;

}

while(lock);

lock = 1;

CS

lock = 0

lock = ~~T~~

P1

while(true)

{

while(lock);

lock=true;

//CS

lock=false;

RS;

}

✗ Mutual Exclusion

✓ Progress

✗ Bounded waiting

P0	P1	P0
while(lock); condit ⁿ false	while(lock); Condit ⁿ false	lock = T in CS

lock = ~~f~~ T

for mutual exclusion not satisfied

checking progress:-

for P0 :- Lock = ~~f~~ ~~T~~ ~~f~~

for P1 :- Lock = ~~f~~ T

checking bounded waiting:-

P0 in CS

P1 waiting

lock = ~~f~~ ~~T~~ ~~f~~ T

`while(lock);` $\Leftrightarrow$ `while(lock == True);` $(\Rightarrow)$ `while(lock  $\neq$  False);`

Solution 2

shared b/w P0 and P1

strict alternation

$P_0 \rightarrow P_1 \rightarrow P_0 \rightarrow P_1 \rightarrow P_0 \dots$

int turn=0;

while(true)

{

while(turn!=0);

CS

turn=1;

RS;

}

P0 can enter into CS when turn is 0

turn = false

while(turn!=false)
C.S.

turn = True

P1

while(true)

{

while(turn!=1);

CS

turn=0;

RS;

}

P1 can enter into CS when turn is 1

✓ Mutual Exclusion

✗ Progress

✓ Bounded waiting

Initially when $turn = 0$, P_1 cannot enter into CS, even if P_0 did not arrive.

↓
no progress



2 mins Summary

Topic

Mutual Exclusion

Topic

Progress

Topic

Bounded Waiting

Topic

Two-Process Solution for Critical Section



Happy Learning

THANK - YOU